AI-Powered Fake Product Review Detection System Using Linguistic and Behavioral Pattern Analysis

Sagar Mishra	Vaibhav Upadhyay	Dr. Athira K
Reg. No. 24BCE2386	Reg. No. 24BCE2095	Faculty, Artificial Intelligence (BCSE306L)
Computer Science and Engineering	Computer Science and Engineering	Project Final Report
Vellore Institute of Technology	Vellore Institute of Technology	

ABSTRACT

Fake product reviews written by paid reviewers, bots and, increasingly, large language models distort purchase decisions and erode trust in e-commerce platforms. Most detectors rely on text sentiment or keyword features alone. They therefore miss fraudulent reviews that imitate genuine sentiment, above all AI-generated ones, and they ignore behavioural evidence such as posting bursts, rating-text inconsistency and reviewer activity. This paper presents a two-layer fake-review detection system that fuses a linguistic layer (TF-IDF n-grams, BERT embeddings and stylometric cues for repetitiveness, generic phrasing and unnatural specificity) with a behavioural layer (reviewer and product burstiness, a rating-text mismatch flag and reviewer-activity statistics). The fused feature vector is classified by Random Forest and XGBoost, with a fine-tuned BERT model as a deep-learning comparison. The system returns a calibrated fake-review probability, a mismatch flag and a reviewer-behaviour dashboard. We evaluate on public Yelp and Amazon-style fake-review datasets with reviewer-disjoint splits, so that scores measure generalisation to unseen reviewers instead of memorisation, and we report precision, recall, F1-score and AUC-ROC against a sentiment-only baseline. The combined model reaches F1 = XX.X and AUC-ROC = 0.XX, against XX.X and 0.XX for the sentiment-only baseline. Ablation and leakage experiments show how much each feature group contributes and how much a random split would overstate performance.

CCS Concepts

• Computing methodologies→Natural language processing • Computing methodologies→Supervised learning by classification • Information systems→Spam detection

Keywords

fake review detection; opinion spam; behavioural features; burstiness; rating-text mismatch; TF-IDF; BERT; XGBoost; reviewer-disjoint evaluation

1. INTRODUCTION

Online reviews are now one of the main inputs to a purchase decision. A shopper comparing two products on Amazon or Flipkart usually reads the star rating and a handful of reviews before paying. That influence creates an incentive to manipulate. Sellers hire paid reviewers, run bot accounts or, more recently, ask a language model to write dozens of fluent five-star reviews in a few seconds. Early work on opinion spam showed that such manipulation is widespread [7], and a study of Yelp found that roughly one restaurant review in six was filtered as suspicious [9]. Every fake review misleads a buyer and, at the same time, tilts competition against honest sellers.

Detecting fake reviews is hard for two reasons. First, a deceptive review can copy the vocabulary and sentiment of a real one, so classifiers built on sentiment or keywords alone are easy to fool. Language models make this worse because their output is grammatical, varied and on topic, and people have difficulty telling it apart from genuine reviews [17]. Second, text is only half of the evidence. A fraud campaign leaves traces in the metadata: many reviews from one account in a single day, a burst of reviews for one product around a launch, or a five-star rating attached to a lukewarm or negative text. Studies on Yelp data reported that such behavioural signals were more effective than n-gram text features [12], but they are rarely combined with modern text representations.

This project builds a detector that uses both kinds of evidence. A **linguistic layer** extracts writing-style features with TF-IDF, BERT embeddings and stylometric measures. A **behavioural layer** engineers features from review metadata: burstiness, rating-text mismatch and reviewer activity. Both feature sets feed one classifier (Random Forest or XGBoost), and a fine-tuned BERT model serves as a deep-learning comparison. The system outputs a fake-review probability for every review, a mismatch flag and a dashboard that shows how a reviewer behaves over time.

We also take evaluation seriously. Reviews written by one person tend to look alike, so a random train/test split puts the same reviewer on both sides and rewards memorisation. We therefore split by reviewer and report the random-split score next to the reviewer-disjoint score. The contributions of this work are:

- **A two-layer architecture** (Fig. 1) that fuses linguistic and behavioural evidence instead of relying on sentiment alone.
- **A defined behavioural feature set**: reviewer and product burstiness, a rating-text mismatch score and flag, and reviewer-activity statistics, each with a formal definition (Section 4).
- **A leakage-aware evaluation** with reviewer-disjoint splits, a sentiment-only baseline, a feature-group ablation and a separate test on computer-generated reviews.
- **A working prototype** that scores uploaded reviews, raises a mismatch flag and presents a reviewer-behaviour dashboard.

Section 2 reviews related work. Sections 3 and 4 describe the system design and methods. Section 5 gives the experimental setup, Section 6 the results, Section 7 limitations, Section 8 ethics and Section 9 the conclusion and future work.

2. LITERATURE REVIEW

2.1 Opinion spam and early detectors

Jindal and Liu introduced the study of opinion spam on a large Amazon crawl. They treated near-duplicate reviews as spam, trained a logistic-regression model on review, reviewer and product features, and showed that spam is common in product reviews [7]. Ott et al. built a gold-standard corpus by paying crowd workers to write deceptive hotel reviews and showed that n-gram and psycholinguistic features with a linear classifier separate them from truthful reviews far better than human judges

do [13]. Li et al. later argued that detectors trained on one domain transfer poorly to another and proposed a more general rule for identifying deceptive reviews [8]. Surveys of the field [3, 11] agree that most published systems use either text or metadata, and that labels, which usually come from a platform filter or from crowdsourcing, are a weak point of the whole area.

2.2 Behavioural and relational signals

Mukherjee et al. examined what the Yelp filter might be doing and found that behavioural features such as the number of reviews posted in a day, the share of extreme ratings and content similarity between a reviewer's own posts were more effective than n-gram features for separating filtered from unfiltered reviews [12]. Rayana and Akoglu went further with SpEagle, which combines review text, metadata and the reviewer-product-review network in a collective inference framework and also released the Yelp datasets that most later work uses [15]. These results support the central idea of this project, that behaviour carries signal text cannot. They predate contextual embeddings and do not treat machine-generated text.

2.3 Representation learning and BERT

TF-IDF weighting [18] remains a strong, interpretable text representation for spam tasks. Contextual models such as BERT [4] produce sentence representations that capture style and semantics beyond word counts, and can be used either as frozen feature extractors or fine-tuned end to end. The Hugging Face Transformers library [19] and Sentence-BERT [16] made such models easy to apply to short texts such as reviews. Tree ensembles, including Random Forest [1] and XGBoost [2], handle mixed numeric features of different scales and are a natural choice for fusing text-derived and metadata-derived features, while SHAP values [10] give a per-prediction explanation of such models.

2.4 AI-generated reviews

Salminen et al. generated fake reviews with a language model, found that people struggle to tell them from real reviews, and showed that trained classifiers can still pick up statistical regularities in the generated text [17]. This line of work is relevant here because generated reviews usually come without any account history. A detector that only looks at text must therefore keep up with every new generator, whereas behavioural evidence, where it exists, does not depend on how the text was produced.

2.5 Rating-text inconsistency

A star rating and the text beside it normally agree: a customer who writes warmly gives four or five stars, and one who complains gives one or two. Disagreement has several innocent causes, such as sarcasm, a mixed review that praises one aspect and criticises another, or a mis-tapped star, but it is also a natural by-product of manipulation, because a paid reviewer is often told the rating to give and writes the text separately, and a text generator rarely sees the rating at all. Lexicon-based sentiment tools such as VADER [6] are fast and need no training data, which makes them suitable for turning the text into a number that can be compared with the rating. We treat the resulting mismatch as one feature among many and not as proof of fraud.

2.6 Evaluation practice

Most of the studies above report accuracy or F1 under random cross-validation. For review data this is optimistic, because reviews by the same person share vocabulary, rating habits and sometimes whole sentences, so a random split lets the classifier recognise a reviewer instead of recognising fraud. The labels also differ between studies: crowd-sourced deception [13] and platform filters [12] measure different things, and results do not transfer cleanly between them [8]. For these reasons we evaluate by reviewer, report the random-split score only as a reference, and describe the limits of the labels in Section 7.

2.7 Research gap

Table 1 summarises representative work. Text-only systems cannot see account behaviour, metadata-based systems rarely use contextual text features, and few studies evaluate with reviewer-disjoint splits or report how a sentiment-only baseline compares. Our system targets this gap by combining (i) linguistic style features, (ii) engineered behavioural features, including a rating-text mismatch signal, and (iii) a leakage-aware protocol that includes a sentiment-only baseline.

Table 1. Representative work and the gap each leaves open.

Work	Cues used	Gap for our setting
Jindal & Liu 2008 [7]	duplicates; review, reviewer, product features	labels from duplicates; no AI text
Ott et al. 2011 [13]	n-grams, psycholinguistic; SVM	text only; no metadata
Mukherjee et al. 2013 [12]	behaviour + n-grams on Yelp	no contextual embeddings
Rayana & Akoglu 2015 [15]	text, metadata, review network	needs whole graph; no BERT
Salminen et al. 2022 [17]	text of machine-generated reviews	text only; no behaviour
This work	TF-IDF, BERT, stylometry + behaviour	reviewer-disjoint test, sentiment-only baseline

3. PROPOSED SYSTEM DESIGN

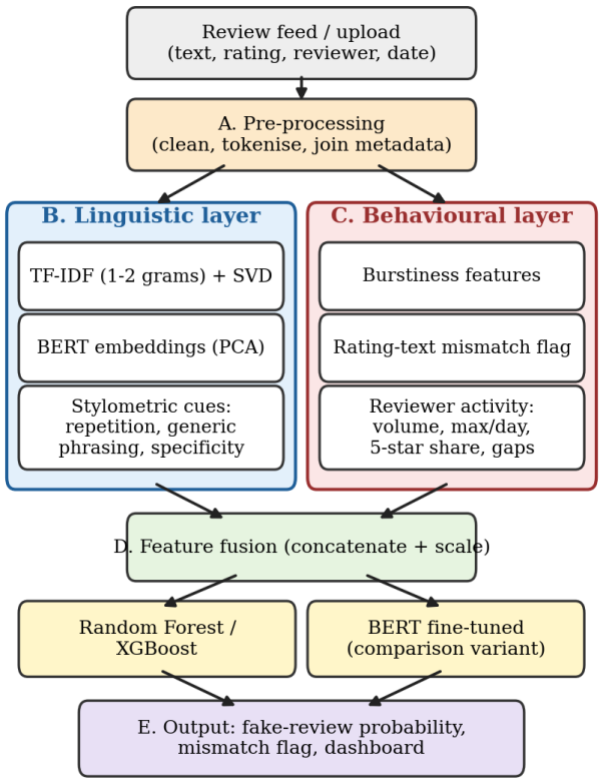


Figure 1. Architecture of the two-layer fake-review detection system. Reviews pass through shared pre-processing, then

through a linguistic and a behavioural layer whose features are fused before classification.

Fig. 1 shows the five modules. Every review enters as a record of text, star rating, reviewer identifier, product identifier and date. After shared pre-processing, the record is analysed in parallel by a linguistic layer and a behavioural layer. Their outputs are concatenated into one feature vector, which is scored by a tree-ensemble classifier. A fine-tuned BERT model runs beside the ensemble as a deep-learning comparison. The final module converts the score into the outputs a platform operator or shopper needs.

3.1 Design goals

Four goals shaped the design:

- **G1, complementary evidence.** Text and behaviour fail in different ways, so both are used and neither is allowed to dominate by construction.
- **G2, interpretability.** A moderator must be able to see why a review was flagged. Hand-designed features and tree models allow this; every alert lists its top features.
- **G3, leakage safety.** Behavioural features are computed without labels and every fitted component uses the training split only, so test scores are honest.
- **G4, light deployment.** Apart from the optional BERT component, the pipeline runs on a CPU and can be embedded in a web service or a browser-extension back end.

3.2 Processing flow

1. A review record arrives from an upload or a feed and is validated (non-empty text, rating in 1 to 5, parsable date).
2. Module A cleans the text and joins the reviewer and product tables.
3. Module B computes the TF-IDF, BERT and stylometric vectors for the review.
4. Module C computes the behavioural features from the review and from the history of its reviewer and product.
5. Module D concatenates the vectors and returns a raw score; the fine-tuned BERT variants return their own scores for comparison.
6. Module E calibrates the score, applies the thresholds, computes the top contributing features and updates the dashboard.

3.3 Module description

Module A: Pre-processing. **Purpose:** turn raw review records into clean, joined data. **Functionality:** lower-casing for the TF-IDF path while keeping the original text for BERT, Unicode and HTML clean-up, removal of exact duplicates, parsing of dates, and a join with reviewer and product tables on their identifiers. **Interaction:** passes the cleaned text to Module B and the metadata to Module C.

Module B: Linguistic layer. **Purpose:** capture how a review is written. **Functionality:** TF-IDF on word unigrams and bigrams followed by truncated SVD, a mean-pooled BERT embedding reduced by PCA, and twelve stylometric measures covering repetitiveness, generic phrasing and unnatural specificity. **Interaction:** outputs a linguistic vector of fixed length to Module D.

Module C: Behavioural layer. **Purpose:** capture how, when and by whom a review was posted. **Functionality:** reviewer burstiness, product burst z-score, rating deviation, the rating-text mismatch score and flag, and reviewer-activity statistics. All features are label-free: no class label is used when computing them. **Interaction:** outputs a behavioural vector to Module D.

Module D: Fusion and classification. **Purpose:** combine both layers into one decision. **Functionality:** concatenation of the two vectors, followed by a Random Forest or XGBoost classifier trained with class weighting. A BERT model fine-tuned on text alone, and a hybrid that appends the behavioural vector to the BERT pooled output, are trained for comparison. **Interaction:** outputs a raw score to Module E.

Module E: Scoring and dashboard. **Purpose:** make the result usable. **Functionality:** isotonic calibration turns the raw score into a probability, thresholds map it to genuine, suspicious or likely fake, and a dashboard (Fig. 2) shows the probability, the mismatch flag, the strongest contributing features and the posting history of the reviewer. **Interaction:** consumes Module D output and writes results to a CSV file; it keeps no review text beyond the session unless the operator exports it.

3.4 User-facing features

The prototype offers four features: (1) review upload or feed, by CSV or by pasting a single review; (2) a fake-review probability score between 0 and 1; (3) a rating-text mismatch flag that tells the user when a star rating and the sentiment of the text disagree; and (4) a reviewer-behaviour dashboard with a posting timeline and burst markers. Fig. 2 gives the layout.

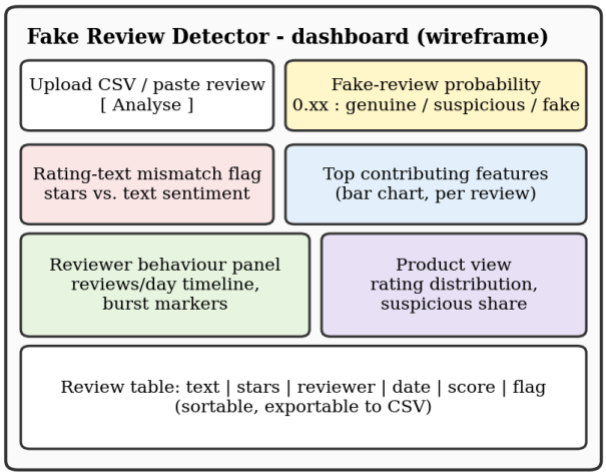


Figure 2. Wireframe of the dashboard. The probability, flag, feature explanation and reviewer panels correspond to the four user-facing features.

4. METHODOLOGY

4.1 Data representation

A review is a tuple $r = (t, s, u, p, d)$: text t, star rating $s \in \{1,\ldots,5\}$, reviewer u, product p and posting date d. The label $y \in \{0,1\}$ is 1 for a fake review and is used only for training and evaluation, never for feature computation. Fig. 3 shows the resulting feature groups.

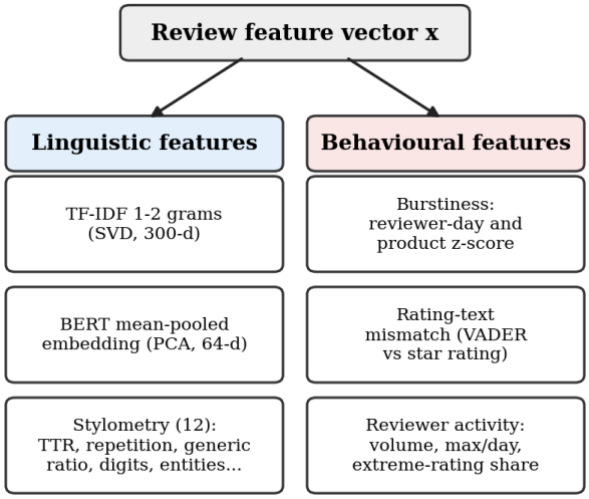


Figure 3. Feature groups. The linguistic vector describes how a review is written; the behavioural vector describes how and when it was posted.

4.2 Text pre-processing

For the TF-IDF path the text is lower-cased, stripped of HTML and control characters and tokenised on word boundaries; stop words are kept because function-word patterns carry stylistic information. For BERT the original casing and punctuation are preserved, because exclamation marks, capitalisation and punctuation habits help distinguish generated from human text. Reviews with fewer than three tokens are dropped.

4.3 TF-IDF features

We use word unigrams and bigrams with minimum document frequency 5 and a vocabulary of at most 50,000 terms. With smoothing, the weight of term w in review r is

$$tfidf(w, r) = tf(w, r) \cdot (\ln[(1 + N) / (1 + df(w))] + 1) \quad (1)$$

where N is the number of training reviews and df(w) the number containing w [18]. Each vector is L2-normalised. The sparse matrix is reduced to 300 dimensions with truncated SVD, fitted on the training split only, so that it can be concatenated with the dense features.

4.4 BERT embeddings

Each review is encoded with the pre-trained bert-base-uncased model [4, 19], truncated to 256 tokens. Let $h_1,\ldots,h_L$ be the final-layer token vectors. The review embedding is the mean over non-padding tokens,

$$e(r) = (1 / L) \cdot \Sigma_{i=1..L}\, h_i \quad (2)$$

a 768-dimensional vector that is reduced to 64 dimensions with PCA fitted on the training split. Mean pooling is used because it is more stable than the [CLS] vector when BERT is not fine-tuned.

4.5 Stylometric features

Twelve measures target the writing traits the abstract highlights. *Repetitiveness* is the repeated-trigram ratio $\rho = 1 -$ (unique trigrams / total trigrams) together with the maximum repeated-word run. *Generic phrasing* is the share of tokens that fall inside a lexicon of stock phrases ("highly recommend", "best product ever", "must buy", "works great"). *Specificity* is the density of concrete detail, (digit tokens + named entities) / L, because genuine reviews tend to mention model numbers, durations and places while generated and paid ones tend to stay vague or, conversely, over-specify in a uniform way. The remaining measures are length in tokens, mean sentence length, type-token ratio, exclamation and capital-letter ratios, first-person pronoun ratio, superlative ratio, and a readability score. Appendix A lists every feature.

4.6 Behavioural features

Reviewer burstiness. Let $n_u(d, w)$ be the number of reviews by reviewer u in the window of w days starting at d. The burst features are the maximum daily count and the maximum weekly count,

$$B_u^{(w)} = \max_d n_u(d, w), \quad w \in \{1, 7\} \quad (3)$$

together with the mean gap between consecutive reviews. Large values mean many reviews in a short span.

Product burst. For product p with daily review counts $c_p(d)$, mean μ_p and standard deviation σ_p, the burst score of a review posted on day d is the z-score of the 7-day count centred on d:

$$z_p(d) = (\bar{c}_p^{(7)}(d) - \mu_p) / (\sigma_p + \varepsilon) \quad (4)$$

A positive z-score means the review arrived during a surge of reviews for that product.

Rating deviation. The absolute distance between the star rating and the product mean computed without the review itself (leave-one-out), $\delta = |s - \bar{s}_p^{(-r)}|$. Reviews that sit far from the consensus, especially at the extremes, are more likely to be planted.

Rating-text mismatch. Sentiment $c(t) \in [-1, 1]$ is the VADER compound score of the text [6]. The star rating is mapped to the same range, $\hat{s} = (s - 3) / 2$. The mismatch score and flag are

$$m = | \hat{s} - c(t) |, \quad flag = 1[\, m > \tau_m \,] \quad (5)$$

where the threshold τ_m is selected on the validation split. A five-star rating attached to a vague or negative-toned text gives a large m and raises the flag. The signed difference $\hat{s} - c(t)$ is also kept, because a high rating with cold text and a low rating with warm text point to different kinds of manipulation.

Reviewer activity. For each reviewer: total number of reviews, share of 5-star and 1-star reviews, number of distinct products reviewed, account span in days between first and last review, and the maximum TF-IDF cosine similarity between any two of the reviewer's own reviews, which exposes copy-and-paste campaigns. Reviewers with a single review receive neutral default values and a "single-review" indicator, which keeps the system usable for cold-start accounts.

4.7 Worked example

The following example is illustrative and is not taken from the data. A reviewer posts the text "Does what it says, nothing special" with five stars. The rating maps to $\hat{s} = (5 - 3) / 2 = 1.0$. Suppose VADER returns $c(t) = 0.0$ for the neutral text. Then $m = |1.0 - 0.0| = 1.0$, which exceeds a threshold of 0.6, so the mismatch flag is raised. If the same reviewer also posted eleven other five-star reviews that day, the maximum daily count in Eq. (3) is 12, and the reviewer-burst features move the review further towards the suspicious band even though the text alone looks harmless.

4.8 Fusion and classifiers

The feature vector concatenates 300 TF-IDF dimensions, 64 BERT dimensions, 12 stylometric measures and the behavioural features:

$$x = [\, x_{tfidf} ; x_{bert} ; x_{sty} ; x_{beh} \,] \quad (6)$$

A Random Forest [1] with 300 trees and balanced class weights and an XGBoost model [2] (depth 6, learning rate 0.05, up to 400 rounds with early stopping, positive-class weight set to the class ratio) are trained on the fused vector. Hyper-parameters are chosen by grid search on the validation split using F1 of the fake class. For the deep-learning comparison, bert-base-uncased is fine-tuned for three epochs (learning rate 2×10^{-5}, batch size 32) on text alone (**BERT-FT**). A hybrid (**BERT+Beh**) concatenates the pooled BERT output with the standardised behavioural vector before the final dense layer.

4.9 Probability score and flags

The raw model score is calibrated with isotonic regression on the validation split so that it behaves as a probability $P(\text{fake} \mid x)$. Two default operating points are used: reviews with $P < 0.4$ are shown as genuine, $0.4 \le P < 0.7$ as suspicious and $P \ge 0.7$ as likely fake. These cut-offs are adjustable, because a platform that moderates automatically needs a higher precision than one that only triages reviews for a human moderator.

4.10 Explainability

Global feature importance comes from impurity (Random Forest) and gain (XGBoost) scores. For individual reviews, TreeSHAP values [10] rank the features that pushed the score up or down; the top five are shown on the dashboard so that a moderator can see why a review was flagged.

4.11 Training procedure

1. Split reviewers into train, validation and test groups (Section 5.2).
2. Fit the TF-IDF vocabulary, SVD and PCA on the training reviews only; transform all three splits.
3. Compute stylometric and behavioural features for every review, using only information from the review's own reviewer and product history.
4. Train the classifiers on the training split with class weighting; select hyper-parameters and the decision threshold on the validation split.
5. Fit isotonic calibration on the validation scores.
6. Evaluate once on the test split and compute bootstrap intervals; repeat for three seeds.

4.12 Complexity

TF-IDF and the stylometric measures are linear in the review length. BERT inference is the dominant cost, roughly quadratic in the token count up to the 256-token limit. Reviewer and product aggregates are computed once per account or product with a single pass over the review table and updated incrementally, so scoring one new review needs only a lookup and a small update. The tree ensembles take milliseconds per review once the features are available.

4.13 Implementation

The prototype is written in Python 3. scikit-learn [14] provides TF-IDF, SVD, PCA, Random Forest, calibration and the grouped splits; XGBoost [2] provides the boosted model; Hugging Face Transformers [19] with PyTorch provides BERT; pandas handles the metadata; VADER [6] provides sentiment; and Streamlit serves the dashboard.

5. EXPERIMENTAL SETUP

5.1 Datasets

Two public sources are used, matching the abstract. The **Yelp fake-review data** of Rayana and Akoglu [15] (YelpChi, YelpNYC and YelpZip) provides review text, rating, reviewer and product identifiers and dates, with labels from the Yelp filter. These are the only data in which every behavioural feature can be computed. The second source is an **Amazon-style fake-review corpus from Kaggle**, in which reviews are labelled original (OR) or computer-generated (CG). It has text, a star rating and a category but no reviewer identifier or date, so it is used for the linguistic layer, the mismatch flag and the AI-generated-review test (Section 6.4). Table 2 gives the statistics.

Table 2. Dataset summary.

Dataset	Reviews	Fake share	Metadata
Yelp (Chi/NYC/Zip) [15]	N	%	full
Kaggle OR/CG corpus	N	%	text, rating

Yelp labels come from a commercial filter, so they are a proxy for ground truth. Mukherjee et al. argue that the filter is a reasonable but imperfect reference [12], and we treat the reported scores accordingly.

5.2 Splits and leakage control

The default protocol is a reviewer-disjoint split: all reviews of a reviewer fall in the same partition, using scikit-learn's grouped splitters [14], with 70% of reviewers for training, 10% for validation and 20% for testing, and five-fold grouped cross-validation as a check. For comparison we also report a conventional random review-level split. The gap between the two shows how much a random split overstates performance. Fitted components (TF-IDF vocabulary, SVD, PCA, scalers, calibration) use the training split only, and every experiment is repeated over three random seeds.

5.3 Baselines and systems compared

Table 3. Systems evaluated.

ID	System	Inputs
S1	Sentiment-only baseline (logistic regression)	VADER score, star rating
S2	TF-IDF + logistic regression	text
S3	TF-IDF + Random Forest	text
S4	Behavioural-only XGBoost	metadata features
S5	BERT-FT (fine-tuned)	text
S6	Linguistic + behavioural, Random Forest	both layers
S7	Linguistic + behavioural, XGBoost	both layers
S8	BERT+Beh hybrid	text + behaviour

S1 stands for the common approach the abstract criticises. S2 and S3 isolate the text representation, S4 isolates the behavioural layer, and S6 to S8 are the proposed combinations.

5.4 Metrics

Fake is the positive class. With true positives TP, false positives FP and false negatives FN, precision is P = TP / (TP + FP), recall is R = TP / (TP + FN), and

$$F1 = 2PR / (P + R) \tag{7}$$

The decision threshold is the one that maximises validation F1. We additionally report AUC-ROC [5], which is threshold-free, and the area under the precision-recall curve, which is more informative when the fake class is rare. A paired bootstrap over

test reviewers (1,000 resamples) gives 95% confidence intervals for the difference between systems.

5.5 Hypotheses

Following the project abstract we test four hypotheses:

- **H1.** The combined linguistic-plus-behavioural model outperforms the sentiment-only baseline on precision, recall, F1 and AUC-ROC.
- **H2.** Adding behavioural features to a text-only model improves F1 on unseen reviewers.
- **H3.** A random review-level split overstates performance compared with a reviewer-disjoint split.
- **H4.** Linguistic features and the mismatch flag detect computer-generated reviews that a sentiment-only model misses.

5.6 Ablations and the AI-generated test

The ablation removes one feature group at a time from the best fused model (TF-IDF, BERT, stylometry, burstiness, mismatch, reviewer activity). The AI-generated test evaluates S1, S2, S5 and a text-plus-mismatch model on the computer-generated reviews of the Kaggle corpus, where behavioural metadata does not exist. Hardware used: CPU / GPU model, RAM.

5.7 Hyper-parameter search

Table 4. Hyper-parameter search space (validation split).

Model	Parameter	Values
Random Forest	trees; max depth	200, 300, 500; none, 20, 40
Random Forest	min samples leaf	1, 2, 5
XGBoost	depth; learning rate	4, 6, 8; 0.03, 0.05, 0.1
XGBoost	subsample; colsample	0.7, 0.9; 0.7, 0.9
Logistic reg.	C	0.1, 1, 10
BERT-FT	learning rate; epochs	2e-5, 3e-5; 2, 3, 4
Mismatch	threshold τ_m	0.4, 0.5, 0.6, 0.8

6. RESULTS

6.1 Main comparison

Table 5 compares the eight systems on the reviewer-disjoint test split (mean over three seeds). Rows run from the sentiment-only baseline to the full models so that the effect of each addition can be read down the table.

Table 5. Reviewer-disjoint test results on the Yelp data (fake = positive class).

System	Prec.	Recall	F1	AUC-ROC
S1 Sentiment-only	--	--	--	--
S2 TF-IDF + LR	--	--	--	--
S3 TF-IDF + RF	--	--	--	--
S4 Behaviour only	--	--	--	--
S5 BERT-FT	--	--	--	--
S6 Ling+Beh, RF	--	--	--	--
S7 Ling+Beh, XGB	--	--	--	--
S8 BERT+Beh	--	--	--	--

Write 3 to 4 sentences here after the runs: which system is best, its F1 and AUC-ROC gain over S1 with the bootstrap interval, whether S6/S7 beat S2/S3 and S5, and whether behaviour alone (S4) is competitive.

6.2 Ablation of feature groups

Table 6. Ablation on the best fused model (change relative to the full model).

Removed group	F1	ΔF1	AUC-ROC
none (full model)	--	0.0	--
TF-IDF	--	--	--
BERT embedding	--	--	--
Stylometric	--	--	--
Burstiness	--	--	--
Rating-text mismatch	--	--	--
Reviewer activity	--	--	--

Write 2 to 3 sentences: which group causes the largest drop when removed, and whether the behavioural groups together matter more than any single text group.

6.3 Effect of the split protocol

Table 7. Random split versus reviewer-disjoint split (best model, F1).

Protocol	F1	AUC-ROC
Random review-level split	--	--
Reviewer-disjoint split	--	--
Gap (leakage)	--	--

Write 2 sentences: how many points the random split overstates F1 and AUC-ROC, and why reviewer-level leakage explains it.

6.4 AI-generated reviews

Table 8. Detection of computer-generated reviews (Kaggle corpus).

System	Prec.	Recall	F1	AUC-ROC
S1 Sentiment-only	--	--	--	--
S2 TF-IDF + LR	--	--	--	--
S5 BERT-FT	--	--	--	--
Text + mismatch, XGB	--	--	--	--

Write 2 to 3 sentences: how well sentiment-only does on generated reviews compared with the linguistic models, and whether the mismatch flag adds anything. State that burstiness and reviewer features were unavailable in this corpus.

6.5 Feature importance

List the five to eight most important features from the XGBoost gain ranking and the SHAP summary, and say in one sentence whether they are mostly behavioural or linguistic.

6.6 Error analysis

After the runs, inspect 20 to 30 false positives and 20 to 30 false negatives of the best model. Write 3 to 4 sentences on the patterns, for example short genuine reviews flagged as generic, genuine bursts around a product launch, or fake reviews posted by accounts with a single review.

6.7 Runtime

Table 9. Mean time per review (ms), batch inference on hardware.

Stage	Time (ms)
Pre-processing + stylometry	--
TF-IDF + SVD	--
BERT embedding	--
Behavioural features	--
Classifier + calibration	--

Write 1 to 2 sentences: which stage dominates (expected to be BERT) and whether the system is fast enough for near-real-time use.

6.8 Hypotheses revisited

Table 11. Outcome of the four hypotheses.

Hypothesis	Evidence	Supported?
H1 combined > sentiment-only	Table 5	Yes / No
H2 behaviour improves text model	Tables 5, 6	Yes / No
H3 random split overstates	Table 7	Yes / No
H4 detects generated reviews	Table 8	Yes / No

7. DISCUSSION AND LIMITATIONS

- **Labels are a proxy.** Yelp labels come from a commercial filter and the OR/CG labels come from how the corpus was built. A model can only be as good as these labels, so scores should be read as agreement with the reference, not as certainty about truth.
- **Behavioural evidence needs metadata.** Reviewer and date information exists on Yelp but not in every corpus. For a brand-new reviewer the behavioural layer has little to say, and the system then depends on the linguistic layer and the mismatch flag.
- **Adaptive adversaries.** Once a platform publishes its signals, fraudsters can spread reviews over time or vary star ratings. Combining several weak signals raises the cost of evasion but does not remove it.
- **Generators keep changing.** A text model trained on today's generated reviews may fail on the next generation of language models, which is why the behavioural layer matters.
- **Sentiment tool limits.** VADER is English-only and handles sarcasm poorly, so the mismatch flag can fire on ironic but genuine reviews.
- **Domain shift.** Models trained on restaurant reviews may transfer imperfectly to electronics or fashion; a cross-domain test is left for future work.
- **Cost of BERT.** Embedding every review takes more compute than TF-IDF; for large platforms the BERT features may be computed offline or distilled.

7.1 Threats to validity

Internal validity. Hyper-parameters and thresholds are chosen on the validation split only, and fitted components never see test reviews, but reviewer aggregates can still reflect label-correlated platform behaviour that we do not control. *External validity.* Results on Yelp restaurants and hotels, and on a single computer-generated corpus, may not hold for other product categories, languages or generators. *Construct validity.* The labels measure agreement with a filter or with a corpus construction rule, not the true intent of each reviewer. We state these limits in the conclusions that we draw from Section 6.

8. ETHICS AND PRIVACY

A false positive removes or discredits an honest customer's review, and a false negative leaves a buyer misled. The system is therefore designed to support a human moderator rather than replace one: scores come with the contributing features, the suspicious band exists to route borderline cases to review, and no review should be deleted on the model score alone. Writers who use English as a second language may produce shorter, plainer reviews that look "generic" to a stylometric model, so performance should be audited across review length, language variety and product category before deployment. Reviewer identifiers are personal data. The prototype uses pseudonymous identifiers, keeps no review text beyond the session unless the operator exports it, and shows reviewer behaviour only to authorised moderators. Finally, a published detector can be studied and evaded, so operators should keep part of the signal set private.

9. CONCLUSION AND FUTURE WORK

We presented a fake-review detector that combines a linguistic layer (TF-IDF, BERT, stylometry) with a behavioural layer (burstiness, rating-text mismatch, reviewer activity) and evaluated it with reviewer-disjoint splits against a sentiment-only baseline. One sentence stating the measured gain in F1 and AUC-ROC from adding behaviour, and one sentence on the AI-generated test. The design makes every decision traceable to named features, runs without a GPU apart from the optional BERT component, and reports the effect of the split protocol so that the numbers can be compared fairly with other work.

Future work includes:

1. A **browser extension** that scores reviews on the page a shopper is reading.
2. **Multilingual support**, replacing VADER and BERT-base with multilingual models so that reviews in Hindi and other Indian languages can be scored.
3. **Reviewer-network graph analysis** to detect coordinated campaigns in which groups of accounts review the same products together.
4. **Cross-domain and cross-platform evaluation**, training on restaurants and testing on products.
5. Combining the pipeline with a dedicated **AI-text detector** and testing it against paraphrased and adversarially edited reviews.

ACKNOWLEDGMENTS

We thank the course faculty of BCSE306L for the guidance and the report template, and the curators of the Yelp and Kaggle fake-review datasets for making them public.

REFERENCES

[1] Breiman, L. 2001. Random forests. *Mach. Learn.* 45, 1 (Oct. 2001), 5-32.

[2] Chen, T. and Guestrin, C. 2016. XGBoost: a scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, CA, August 13-17, 2016). KDD '16. ACM, New York, NY, 785-794.

[3] Crawford, M., Khoshgoftaar, T. M., Prusa, J. D., Richter, A. N., and Al Najada, H. 2015. Survey of review spam detection

using machine learning techniques. *J. Big Data* 2, 1 (Oct. 2015), Article 23.

[4] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. 2019. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (Minneapolis, MN, June 2-7, 2019). ACL, 4171-4186.

[5] Fawcett, T. 2006. An introduction to ROC analysis. *Pattern Recognit. Lett.* 27, 8 (Jun. 2006), 861-874.

[6] Hutto, C. J. and Gilbert, E. 2014. VADER: a parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the 8th International AAAI Conference on Weblogs and Social Media* (Ann Arbor, MI, June 1-4, 2014). ICWSM '14. AAAI Press, 216-225.

[7] Jindal, N. and Liu, B. 2008. Opinion spam and analysis. In *Proceedings of the 2008 International Conference on Web Search and Data Mining* (Palo Alto, CA, February 11-12, 2008). WSDM '08. ACM, New York, NY, 219-230.

[8] Li, J., Ott, M., Cardie, C., and Hovy, E. 2014. Towards a general rule for identifying deceptive opinion spam. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics* (Baltimore, MD, June 22-27, 2014). ACL, 1566-1576.

[9] Luca, M. and Zervas, G. 2016. Fake it till you make it: reputation, competition, and Yelp review fraud. *Manage. Sci.* 62, 12 (Dec. 2016), 3412-3427.

[10] Lundberg, S. M. and Lee, S.-I. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (Long Beach, CA, December 4-9, 2017). NIPS '17. Curran Associates, 4765-4774.

[11] Mohawesh, R., Xu, S., Tran, S. N., Ollington, R., Springer, M., Jararweh, Y., and Maqsood, S. 2021. Fake reviews detection: a survey. *IEEE Access* 9 (2021), 65771-65802.

[12] Mukherjee, A., Venkataraman, V., Liu, B., and Glance, N. 2013. What Yelp fake review filter might be doing? In *Proceedings of the 7th International AAAI Conference on Weblogs and Social Media* (Cambridge, MA, July 8-11, 2013). ICWSM '13. AAAI Press, 409-418.

[13] Ott, M., Choi, Y., Cardie, C., and Hancock, J. T. 2011. Finding deceptive opinion spam by any stretch of the imagination. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics* (Portland, OR, June 19-24, 2011). ACL, 309-319.

[14] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. 2011. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* 12 (Oct. 2011), 2825-2830.

[15] Rayana, S. and Akoglu, L. 2015. Collective opinion spam detection: bridging review networks and metadata. In *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Sydney, Australia, August 10-13, 2015). KDD '15. ACM, New York, NY, 985-994.

[16] Reimers, N. and Gurevych, I. 2019. Sentence-BERT: sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019* (Hong Kong, China, November 3-7, 2019). ACL, 3982-3992.

[17] Salminen, J., Kandpal, C., Kamel, A. M., Jung, S.-G., and Jansen, B. J. 2022. Creating and detecting fake reviews of online products. *J. Retail. Consum. Serv.* 64 (Jan. 2022), Article 102771.

[18] Salton, G. and Buckley, C. 1988. Term-weighting approaches in automatic text retrieval. *Inf. Process. Manag.* 24, 5 (1988), 513-523.

[19] Wolf, T., Debut, L., Sanh, V., et al. 2020. Transformers: state-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (Online, November 16-20, 2020). ACL, 38-45.

APPENDIX A: FEATURE DICTIONARY

Table 10. Features used by the fused model.

Group	Feature	Definition
Linguistic	tfidf_svd (300)	TF-IDF 1-2 grams, SVD
Linguistic	bert_pca (64)	mean-pooled BERT, PCA
Stylometric	n_tokens	review length in tokens
Stylometric	avg_sent_len	mean tokens per sentence
Stylometric	ttr	types / tokens
Stylometric	rep_trigram	1 - unique/total trigrams
Stylometric	max_word_run	longest repeated-word run
Stylometric	generic_ratio	stock-phrase tokens / L
Stylometric	specificity	(digits + entities) / L
Stylometric	excl_ratio, caps_ratio	"!" and capital share
Stylometric	first_person	I/me/my share
Stylometric	superlative	best/worst/perfect share
Stylometric	readability	Flesch reading ease
Behaviour	rev_max_day, rev_max_week	Eq. (3)
Behaviour	rev_mean_gap	mean days between reviews
Behaviour	prod_burst_z	Eq. (4)
Behaviour	rating_dev	leave-one-out deviation
Behaviour	mismatch, mismatch_flag	Eq. (5); signed copy
Behaviour	rev_count, distinct_prod	reviewer volume
Behaviour	share_5star, share_1star	extreme-rating share
Behaviour	account_span	last - first review (days)
Behaviour	self_sim_max	max TF-IDF cosine, own reviews
Behaviour	single_review	cold-start indicator

APPENDIX B: DASHBOARD WALKTHROUGH

1. Open the dashboard and upload a CSV with at least the columns text, rating, reviewer_id, product_id and date, or paste a single review.
2. Press Analyse. The table fills with one row per review, showing the fake-review probability and the label genuine, suspicious or likely fake.
3. Select a row. The side panels show the rating-text mismatch flag, the five features that contributed most to the score, and the posting timeline of the reviewer with burst markers.
4. Switch to the product view to see the rating distribution and the share of suspicious reviews for that product.
5. Export the table to CSV for moderator follow-up.

APPENDIX C: GENERIC-PHRASE LEXICON (EXCERPT)

The generic-phrase lexicon behind the stylometric feature *generic_ratio* contains stock expressions that carry little product-specific information, for example: "highly recommend", "best product ever", "must buy", "works great", "love it", "five stars", "exceeded my expectations", "perfect gift", "great quality", "would buy again", "fast shipping" and "worth every penny". The list is extended with the most frequent bigrams and trigrams that

appear in reviews of many unrelated products in the training split, which keeps it data-driven and avoids hand-picking phrases that happen to correlate with the label.